リサーチレポート: Polymarketにおける裁定取引

1. note.mdの内容

> polymarketにおける裁定取引の手法や実績について、リサーチを行え。 実際に作れるように説明せよ。

---

2. 問いの明確化

背景・文脈

Polymarketは世界最大の予測市場プラットフォームであり、2025年に$215億の取引量を記録した。裁定取引（アービトラージ）は理論的にリスクフリーな利益機会として注目されており、実際にボットによる大規模な利益抽出が学術研究で実証されている。単なる概念理解ではなく、「自分でも実装できる」レベルの実践知を求めている。

核心的な関心事

- Polymarketで実際に裁定取引は成立しているか、どの程度の利益が実績として出ているか？
- 具体的にどのような手法が存在し、それぞれどのように機能するか？
- PythonなどでAPIを使った裁定ボットを実際に作るにはどうすればよいか？

暗黙の前提・仮説

- 予測市場における裁定はリスクゼロに近いという前提があるが、実際には解決リスクや手数料が存在する
- 自分でボットを実装・運用することを想定している（単なる概念説明ではなく実装ガイドが必要）
- 米国規制の制限（US居住者はPolymarket利用不可）は考慮済みと推定

---

3. 論点設定

#	論点	調査の方向性
1	Polymarketの市場構造はどのような価格非効率を生み出しており、裁定機会の構造的発生源は何か？	CLOBの仕組み、流動性分布、価格更新ラグを調査
2	実績ベースで裁定取引はどの程度の利益を上げてきたか？手法別の収益性と競合状況は？	学術論文（IMDEA研究）・事例・ボット運用者の報告を分析
3	主要な裁定手法（市場内・クロスプラットフォーム・レイテンシー）の仕組みと実装上の要件は？	各手法のメカニズム・必要インフラ・コード実装パターンを整理
4	手数料・ガス代・解決リスクを考慮したとき、裁定取引の実際の採算ラインと限界はどこか？	手数料体系・Polygon gas・oracle解決プロセスを定量化

---

4. リサーチ結果

論点1: Polymarketの市場構造と価格非効率の発生源

ファクト

- Polymarketは**CLOB（中央集権型注文板）**方式を採用。Polygon（旧Matic）ブロックチェーン上で決済し、USDCを担保とした条件付きトークンを取引する [1][2]
- 2025年の取引量は**$215億**（前年比+139%）、アクティブトレーダーは**45万人超**、累計オンチェーントランザクションは**9,500万件** [3][4]
- 各市場はYES/NOの2種類のトークンで構成され、解決時にどちらか一方が$1.00に、もう一方が$0.00に確定する [1]
- 市場参加者の情報非対称・遅い手動トレーダーの存在・リアルタイム価格更新の遅延が価格非効率の主因 [5]
- IMDEA Networks Instituteの研究（8,600万件の取引を分析）では、**「確率の森」**と呼ぶ相互依存した市場ネットワーク内に構造的な裁定機会が発生することを実証 [6]

分析・考察

CLOBは理論的に効率的な価格発見を促すが、Polymarketには以下の特有の非効率が存在する。①ロングテールの低流動性市場での価格更新遅延、②スポーツ・選挙な

ど互いに関連するイベントの同時存在、③24時間稼働しリアルタイム情報を処理できる人間の少なさ。これらが裁定機会の構造的な発生源となっている。

小括

Polymarketの市場構造自体が価格非効率を内包しており、特に「関連市場間の論理的整合性の欠如」と「リアルタイム処理能力の非対称性」が裁定機会の主因。規模拡大にもかかわらず機会は継続的に存在している。

論点2: 裁定取引の実績と収益性

ファクト

- IMDEA Networks Instituteによる学術研究（2025年公表、arXiv: 2508.03474）が2024年4月〜2025年4月の1年間で**約$4,000万の裁定利益**を実証 [6][7]
- 上位3ウォレットの合計利益は**約$420万**。最上位の単独トレーダーは**4,049回の取引で$201万**を獲得 [6]
- 2026年時点でも、クロスプラットフォーム裁定では**月12〜20%のリターン**が報告されているが、これはスプレッドが大きい機会に限定 [8]
- 1日に運用されたボット（0xalberto）が**1日で$764の利益**を得た事例あり（2025年12月） [9]
- 裁定機会の平均持続時間は2024年の**12.3秒**から2026年には**2.7秒**に短縮。裁定利益の**73%がサブ100ミリ秒のボット**に独占されている [5]
- 政治市場（2024年米大統領選）とスポーツ市場が裁定機会・利益ともに最大のカテゴリ [6]

分析・考察

$4,000万という数字は印象的だが、1年間の全市場合計であり、個人レベルでの期待値は大幅に下がる。機会の持続時間が2.7秒まで短縮されたことは、手動トレーダーが介入できる余地がほぼ消失したことを意味する。一方で、低頻度・高スプレッドの機会（特に流動性の低いロングテール市場）は依然として機械的に検索すれば捕捉可能。参入者が増えるほど機会は縮小するが、Polymarketの市場数が急増（数万市場）しているため、絶対的な機会量は増加傾向にある。

小括

裁定取引は実績として大規模に成立しており、少なくとも2024〜2025年は個人ボットでも数百万ドル規模の利益が可能だった。しかし競争激化により、2026年以降は低レイテンシーインフラなしでは主要な機会へのアクセスが困難になっている。

論点3: 主要手法の仕組みと実装要件

ファクト

【手法A】市場内再均衡裁定（Intra-market Rebalancing Arbitrage） - YES + NO の合計価格が$1.00を下回る場合、両方を購入すると確実に利益 [5][10] - 例: YES = $0.45、NO = $0.52 → 合計$0.97で購入し、$1.00で解決。$0.03/セットの利益 - 手数料（2%の勝者手数料）を考慮すると、スプレッドが**最低3%以上**必要 [11] - 実装難易度: 低〜中。価格ポーリング + 注文実行で実現可能

【手法B】コンビナトリアル裁定（Combinatorial Arbitrage） - 論理的に関連する複数市場間の価格矛盾を利用 [6] - 例: 「トランプが2028年選挙で勝利」= 35%、「共和党が2028年選挙で勝利」= 30% → 前者が後者を包含するため、前者の方が高くなければ矛盾 - 実装難易度: 高。市場間の論理関係を自動検出するNLP/グラフアルゴリズムが必要

【手法C】クロスプラットフォーム裁定（Cross-platform Arbitrage） - Polymarket（分散型）とKalshi（規制下）で同一イベントの価格差を利用 [8][12] - 例: Polymarketで「A国がB国に宣戦布告」= 60%、Kalshiで= 54% → Polymarket NO + Kalshi YES で購入、合計$0.94でリスクヘッジ - 実際の事例: 2026年2月のLA市長選で Kalshi YES 58¢ + Polymarket NO 35¢ = 93¢ → 7.53%のリターン [8] - 最大のリスク: 両社の「解決基準」が異なる場合（2024年政府閉鎖事例: Polymarketは「OPM閉鎖発表」、Kalshiは「24時間超の実際の閉鎖」が条件で、真逆の解決が生じた） [12] - 実装難易度: 中。両プラットフォームのAPIを並列監視するスクリプトが必要

【手法D】レイテンシー裁定（Latency Arbitrage） - BinanceなどのリアルタイムBTC価格とPolymarketの価格更新ラグ（30〜90秒）を利用 [13] - Polymarketが2025年に動的Taker手数料（最大1.80%）を導入したことで事実上終了 [11] - 現在は Rust + 低レイテンシーサーバーを使っても採算が取りにくい状態

【技術実装】Python CLOBクライアント

```
# インストール
# pip install py-clob-client

from py_clob_client.client import ClobClient
from py_clob_client.clob_types import OrderArgs, OrderType, Side

# 初期化（Polygon mainnet: chain_id=137）
client = ClobClient(
    host="https://clob.polymarket.com",
    key="YOUR_PRIVATE_KEY",
    chain_id=137,
    funder="YOUR_WALLET_ADDRESS"
)

# 1. 注文板の取得（認証不要）
order_book = client.get_order_book("TOKEN_ID")
print(order_book.bids)  # 買い注文
print(order_book.asks)  # 売り注文

# 2. 裁定検出（YES + NO < $1.00）
yes_best_ask = float(order_book.asks[0].price)  # YESの最安値
no_book = client.get_order_book("NO_TOKEN_ID")
no_best_ask = float(no_book.asks[0].price)

spread = 1.0 - (yes_best_ask + no_best_ask)
if spread > 0.03:  # 3%以上のスプレッドで実行
    # 3. 両サイド同時発注（リミット注文）
    yes_order = client.create_order(OrderArgs(
        price=yes_best_ask,
        size=100,  # $100分
        side=Side.BUY,
        token_id="YES_TOKEN_ID"
    ))
    client.post_order(yes_order, OrderType.GTC)

    no_order = client.create_order(OrderArgs(
        price=no_best_ask,
        size=100,
```

```
        side=Side.BUY,
        token_id="NO_TOKEN_ID"
    ))
    client.post_order(no_order, OrderType.GTC)
```

主要なGitHubリポジトリ [14][15][16]: - `Polymarket/py-clob-client`: 公式Python SDK - `0xalberto/polymarket-arbitrage-bot`: 単一・複数市場裁定ボット - `ImMike/polymarket-arbitrage`: Polymarket + Kalshi間のクロスプラットフォームボット - `CarlosIbCu/polymarket-kalshi-btc-arbitrage-bot`: BTC 1時間価格市場専用ボット

分析・考察

手法AとCが現実的な個人実装の範囲内。手法Bはグラフ理論・NLPが必要で難易度高。手法Dは現実的ではない。実装の最大の障壁は「同時発注の非原子性」——YES側は約定してNO側は失敗するリスクがあり、手動リスク管理が必須。WebSocketを使ったリアルタイム価格監視 + 失効時間付き注文（GTD）の組み合わせが実用的なアプローチ。

小括

市場内再均衡裁定（手法A）が最も実装しやすく、公式PythonSDK（py-clob-client）とオープンソースの参照実装が揃っているため、数日以内にプロトタイプを作成可能。クロスプラットフォーム裁定（手法C）は解決リスクの管理が追加で必要。

論点4: 手数料・リスクを考慮した採算ラインと限界

ファクト

- **勝者手数料**: 利益の2%（例: $100の勝利で$2の手数料） [11]
- **Taker手数料（動的）**: 最大1.80%（高頻度クリプト市場）。確率50%付近で約1.56%、極端な確率（5%や95%）では0%に近づく [11]
- **ガス代**: Polygonで通常0.01〜0.1セント。混雑時は$0.50〜$1.00まで上昇するが、Polymarketのプロキシウォレット経由では実質的に無料（プラットフォームが代行） [11]
- **採算ラインの目安**: $100取引で3.0%以上、$500取引で2.5%以上、$1,000以上の取引で2.2%以上のスプレッドが必要 [5]
- **解決リスク**: UMA oracleによる解決プロセスで異議申立てが発生した場合、資本が数日〜数週間ロック。異議申立てには750 USDCのボンドが必要 [11]

- **クロスプラットフォームの解決相違リスク**: 2024年米政府閉鎖事例では、Polymarketは「閉鎖なし」と解決し、Kalshiは「閉鎖あり」と解決する事態が発生。同一イベントのはずが真逆の結果になったケースが実在 [12]
- **流動性リスク**: 大口注文時はスリッページが発生。$1,000超の注文で注文板が薄い市場では期待価格から大きくずれる可能性 [5]

分析・考察

手数料体系を見ると、Polymarketは「頻繁に取引するアービトラージャーから収益を得る」設計に移行しつつある（動的Taker手数料）。採算ラインはトレード規模に依存するため、大資本ほど有利。個人がリスク許容できる最大のポジションは、一般的に流動性の20〜30%以内に留めるべきで、実際には数百ドル規模のトレードが現実的。解決リスクの管理が最重要課題であり、「明確・一義的な解決基準を持つ市場のみを対象とする」フィルタリングがボット設計の核心となる。

小括

手数料と解決リスクを考慮すると、スプレッドが3%未満の機会はほぼ採算が合わない。採算ラインを上回る機会は1日5〜15回程度存在するが、その73%は超高速ボットに先取りされる。個人が勝てる余地は「大手ボットが監視しにくいニッチな市場」と「解決直前の価格収束」に絞られる。

---

5. まとめ

総合所見

Polymarketにおける裁定取引は実際に$4,000万規模の利益が生まれていることが学術的に実証されており、ツール（公式Python SDK + オープンソースボット）も整備されているため技術的な参入障壁は低い。しかし、競争の激化（機会持続時間2.7秒・利益の73%が超高速ボット独占）と動的手数料の導入により、「ツールを作ればすぐ儲かる」状況ではない。個人が現実的に勝負できるのは、大手ボットがカバーしきれない低流動性ニッチ市場のクロスプラットフォーム裁定か、解決間際の価格収束狙いのポジション取りに限られる。

確度の高い結論

- Polymarket市場内の再均衡裁定（YES+NO < $1.00）は確実に存在し、公式Python SDKで数日以内に検出ボットを実装可能

- クロスプラットフォーム裁定（vs Kalshi）は採算が取れる機会が存在するが、解決基準の相違リスクが最大の落とし穴
- 手数料（勝者手数料2% + Taker手数料最大1.80%）を考慮すると、スプレッドが3%未満の機会は損失になる
- 個人実装でも1日$100〜$500規模の利益は可能（流動性の低い市場を対象にした場合）

まだ不確実な点

- 動的Taker手数料の将来的な変更方向（さらに引き上げられるかどうか）
- Polymarket自身のトークン発行後の市場構造変化（インセンティブ設計が変わる可能性）
- 規制強化（CFTC等）がKalshiとPolymarketの両方を制約した場合のクロスプラットフォーム戦略への影響

ネクストアクション

1. `pip install py-clob-client` で環境構築し、まず認証不要の `get_order_book()` でリアルタイム価格を取得するスクリプトを作成
2. `0xalberto/polymarket-arbitrage-bot` を参照してYES+NO価格の監視ロジックを実装
3. 実際の取引前に、小額（$10〜$50）でテストし手数料と約定タイミングを確認
4. 解決基準が明確な市場（スポーツの公式試合結果・選挙の公式当選者等）のみを対象にするフィルタを実装

---

6. 追加の論点

#	追加論点	なぜこの問いが重要か
1	Polymarket自身がトークンエコノミー（POLY）を導入した場合、アービトラージャーへのインセンティブ設計はどう変化し、新たな機会が生まれるか？	トークン報酬が加わることで採算ラインが下がり、今は不採算な機会が採算化される可能性がある。プラットフォームの収益設計を理解することで先読みできる
2	AIエージェント（LLM + tool use）がポリシー判断を含む自律的なアービ	Polymarket公式がAIエージェントフレームワークを公開しており、LLMベースの市場分析が競合として登場している。個人ボッ

#	追加論点	なぜこの問いが重要か
	トラージャーとして市場参加した場合、市場効率はどこまで高まるか？	トが勝負できる余地がさらに縮小するかの判断材料になる
3	クロスプラットフォーム裁定において、Polymarket（無規制）とKalshi（CFTC規制）の解決基準の相違を事前に定量的に予測する方法はあるか？	解決リスクは最大の損失要因だが、解決基準の「曖昧さスコア」を事前評価できれば、採算の見込める機会と見込めない機会を事前に分類できる
4	Polymarketの注文板データをオンチェーンで直接取得し、フロントランニング的なアービトラージ（MEV的手法）を実行することは技術的に可能か？	PolygonのMEVは困難だが、特定の大口注文の検出→先回り注文という手法の可能性と、倫理的・規約上の問題を同時に検証する価値がある